

Dynamic Graph

CS3014 - Estructura de Datos Avanzados

Luciano A. Romero Calla

lromeroc@utec.edu.pe

2026-1

Overview

1. Warming Up
2. The Dynamic Tree Problem
3. Structural Tool: Preferred Paths
4. Heavy-Light Decomposition (HLD)
5. Summary

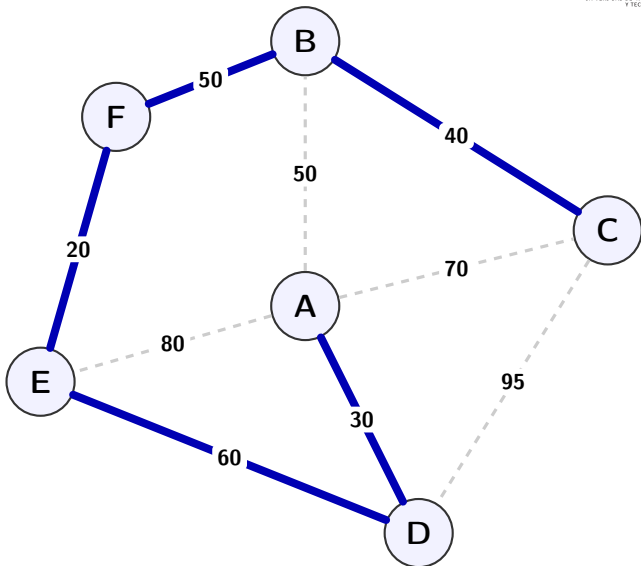
1.

Warming Up

Warming Up

Problem

Remember the problem of Imperial Roads. Now the king may want to keep a chosen road. How does this change your previous solution?



2.

The Dynamic Tree Problem

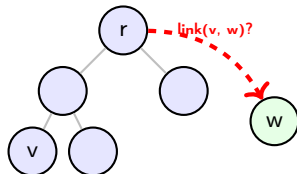
The Dynamic Tree Problem

The Challenge: Maintain a forest of rooted trees under:

- ▶ **Structural updates** (Link/Cut).
- ▶ **Connectivity queries** (FindRoot).
- ▶ **Path aggregates** (Min/Max/Sum).

Efficiency Goal

$O(\log n)$ amortized time per operation.



3.

Structural Tool: Preferred Paths

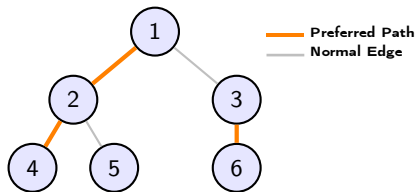
3.1 Internal Representation: Auxiliary Trees

3.2 The Foundation: *access(v)*

Structural Tool: Preferred Paths

We decompose the tree into **vertex-disjoint paths** based on access patterns.

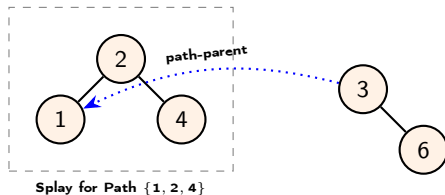
- ▶ **Preferred Child:** The child c of v such that the last access in v 's subtree was in c 's subtree.
- ▶ **Preferred Edge:** Connection to preferred child.
- ▶ **Preferred Path:** Maximal chain of preferred edges.



Internal Representation: Auxiliary Trees

Each preferred path is stored in an **Auxiliary Tree** (Splay Tree).

- ▶ **Order:** Keyed by **depth** in the original tree.
- ▶ **Left Subtree:** Ancestors (smaller depth).
- ▶ **Right Subtree:** Descendants (larger depth).



The Foundation: $\text{access}(v)$

This operation forces the root-to- v path to become a single preferred path.

1. **Splay** v to the root of its Aux-Tree.
2. **Discard** v 's right child (it becomes a separate path).
3. **Climb** via path-parent to $w = \text{pp}(v)$.
4. **Splay** w and make v its new **right child**.
5. **Repeat** until the represented root is reached.

Effect

After $\text{access}(v)$, node v is at the root of a Splay tree containing exactly the path from v to the tree root.

4.

Heavy-Light Decomposition (HLD)

4.1 HLD: Why the Logarithmic Bound?

4.2 Complexity Proof Sketch

Heavy-Light Decomposition (HLD)

To prove $O(\log n)$, we use HLD as an **analysis tool** (not part of the implementation).

Definition

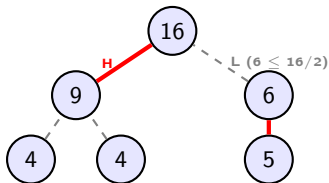
Let $\text{size}(v)$ be the number of nodes in v 's subtree.

- ▶ Edge (u, v) is **Heavy** if $\text{size}(v) > \frac{1}{2}\text{size}(u)$.
- ▶ Edge (u, v) is **Light** otherwise.

Crucial Lemma: Any path from root to node contains at most $\log_2 n$ light edges.

HLD: Why the Logarithmic Bound?

Moving down a **Light Edge** reduces the subtree size by at least half.



$$\text{Max Light Edges} \approx \log_2(16) = 4$$

Complexity Proof Sketch

The cost of access (v) is the number of preferred-edge changes.

Preferred-Child Changes

- ▶ **Light Changes:** Accessing a light child. Only $\log n$ exists on any path.
- ▶ **Heavy Changes:** Accessing a heavy child. Can be many, but they *create* light edges for future operations.

Total Cost

By potential function analysis (based on HLD), the number of heavy-to-light transitions is bounded. Total: $O(\log n)$ amortized.

5.

Summary

Summary

- ▶ **Link-Cut Trees** solve dynamic forest problems in $O(\log n)$.
- ▶ **Key Subroutine:** `access(v)` handles the path logic.
- ▶ **Analysis:** Heavy-Light Decomposition provides the amortized guarantee.

Operation	Mechanism	Complexity
Access(v)	Climbing Aux Trees	$O(\log n)$
Link(v, w)	Access + pp link	$O(\log n)$
Cut(v)	Access + remove left	$O(\log n)$
PathSum(v)	Access + Splay Aggregate	$O(\log n)$

Thanks



References

Acknowledgements

The course slides are based on the lectures from previous editions and on similar lectures elsewhere.
List of credits: Erick Demaine, Keith Schwarz